
e-Portfolio

Objekt-Orientierte Programmierung
WiSe 2025/2026

Kleine Graphen-Bibliothek

Name:	Melchi Dangui
Gruppennummer:	287
Übrige Gruppenmitglieder:	Leyla Louache, Mohamed Eltantawy
Sprecherin der Gruppe:	Leyla Louache
Studiengang:	M.Sc. Data Science
Abgabedatum:	01. Juni 2026

Lehrveranstaltung: Objekt-Orientierte Programmierung
Dozenten: Dr. M. Giar, Prof. Dr. C. Heiliger
Universität: Justus-Liebig-Universität Gießen
Modul: 07-BAI-08 / 07-MDS-04

Contents

1	Zusammenfassung des Logbuchs	2
1.1	Aufgabe und Ziel im Projektkontext	2
1.2	Implementierungsansätze und Design-Entscheidungen	2
1.2.1	Graphklassen mit erweiterter Schnittstelle	2
1.2.2	Duck Typing über <code>detail::get_target()</code> in den Headers . . .	3
1.2.3	Zusammenhangskomponenten mit drei Hilfsfunktionen	4
1.3	Tests	4
1.4	Schwierigkeiten und Grenzen	5
1.5	Zwischenstand und Ausblick	6
2	Einschätzung der Gruppenmitglieder	7
3	Quellen und Hilfsmittel	9
A	Logbuch	11

1 Zusammenfassung des Logbuchs

1.1 Aufgabe und Ziel im Projektkontext

Meine konkreten Beiträge zur Graphenbibliothek umfassen:

- **Header-Dateien:** `directed_graph.hpp`, `undirected_graph.hpp`, `traversal.hpp`, `connectivity.hpp` — inklusive erweiterter Schnittstelle (`has_edge`, `edge_count`, `degree`, `in_degree`, `empty`, `to_graphviz`, `count_components`, `component_ids`, `is_connected`)
- **Implementierungen:** `src/directed_graph.cpp`, `src/undirected_graph.cpp`
- **Unit-Tests:** `test_bfs.cpp` (5 Testfälle), `test_dfs.cpp` (5 Testfälle), `test_topological_sort.cpp` (4 Testfälle), `test_connected_components.cpp` (5 Testfälle)
- **Beispielprogramme:** `example_bfs.cpp`, `example_dfs.cpp`, `example_topological_sort.cpp`, `example_connected_components.cpp`

Ziel war eine modulare, typsichere Bibliothek, die dem *DRY*-Prinzip [2] und dem *Single Responsibility Principle* [3] folgt, Duck Typing für Template-Algorithmen ermöglicht und als externes CMake-Projekt eingebunden werden kann.

1.2 Implementierungsansätze und Design-Entscheidungen

Graphklassen mit erweiterter Schnittstelle

Die Klassen `DirectedGraph` und `UndirectedGraph` gehen über die Mindestanforderungen hinaus: Neben `add_vertex()`, `add_edge()` und `neighbors()` bieten sie `has_edge()`, `degree()`, `in_degree()` (nur directed), `edge_count()`, `empty()`, `edges()` und `to_graphviz()`. Diese Entscheidung folgt dem Prinzip, eine vollständige und testbare Schnittstelle anzubieten.

```

1 std::size_t DirectedGraph::in_degree(vertex_type v) const {
2     if (!has_vertex(v))
3         throw std::out_of_range("Invalid vertex");
4     std::size_t count = 0;
5     for (const auto& neighbors : adj_)
6         for (auto u : neighbors)
7             if (u == v) ++count;
8     return count;
9 }
```

Listing 1: `directed_graph.cpp`: `in_degree()` zählt eingehende Kanten durch Iteration über alle Adjazenzlisten — $O(V + E)$, aber korrekt und ohne zusätzliche Datenstruktur.

```

1  std::vector<std::pair<vertex_type, vertex_type>>
2  UndirectedGraph::edges() const {
3      std::vector<std::pair<vertex_type, vertex_type>> result;
4      for (vertex_type a = 0; a < adj_.size(); ++a)
5          for (auto b : adj_[a])
6              if (a < b) // jede Kante nur einmal
7                  result.emplace_back(a, b);
8      return result;
9  }
10
11 std::size_t UndirectedGraph::edge_count() const {
12     std::size_t count = 0;
13     for (const auto& neighbors : adj_)
14         count += neighbors.size();
15     return count / 2; // beide Richtungen
16                       gespeichert

```

Listing 2: `undirected_graph.cpp`: `edges()` gibt jede Kante nur einmal zurück (Bedingung `a < b`). `edge_count()` teilt durch 2, da jede Kante in beide Richtungen gespeichert ist.

Duck Typing über `detail::get_target()` in den Headers

Ein zentrales Design-Problem: BFS, DFS und Zusammenhangskomponenten sollen sowohl mit ungewichteten Graphen (`Nachbar = std::size_t`) als auch mit gewichteten Graphen (`Nachbar = Edge-Struct` mit `.to`-Feld) funktionieren. Die Lösung ist ein namespace `detail`-Helper mit Überladungen (Listing 3), der ohne Vererbung oder Concepts auskommt.

```

1  namespace detail {
2
3      // Fuer gewichtete Graphen: gibt edge.to zurueck
4      template <typename T>
5      auto get_target(const T& neighbor) -> decltype(neighbor.to) {
6          return neighbor.to;
7      }
8
9      // Fuer ungewichtete Graphen: gibt v direkt zurueck
10     inline std::size_t get_target(std::size_t v) {
11         return v;
12     }
13
14 } // namespace detail

```

Listing 3: `traversal.hpp` und `connectivity.hpp`: `get_target()` wählt per Overload Resolution zur Compile-Zeit, ob der Nachbar ein `size_t` oder ein `Edge-Struct` ist.

Damit kann BFS mit `detail::get_target(neighbor)` den Zielknoten extrahieren,

egal welcher Graphtyp übergeben wird — ohne Template-Spezialisierung oder Vererbung. Diese Entscheidung hält den Code kompakt und vermeidet Code-Duplikation (DRY).

Zusammenhangskomponenten mit drei Hilfsfunktionen

`connectivity.hpp` bietet drei komplementäre Funktionen: `connected_components()` liefert die vollständige Struktur, `count_components()` gibt die Anzahl zurück, und `component_ids()` weist jedem Knoten eine Komponenten-ID zu. `is_connected()` prüft den Spezialfall eines leeren Graphen separat.

```

1  template <typename Graph>
2  std::vector<int> component_ids(const Graph& g) {
3      using V = typename Graph::vertex_type;
4      std::vector<int> ids(g.size(), -1);
5      int current_id = 0;
6
7      for (V start = 0; start < static_cast<V>(g.size()); ++start)
8      {
9          if (ids[start] != -1) continue;
10         std::vector<V> stack{start};
11         ids[start] = current_id;
12         while (!stack.empty()) {
13             V v = stack.back(); stack.pop_back();
14             for (const auto& neighbor : g.neighbors(v)) {
15                 V u = detail::get_target(neighbor);
16                 if (ids[u] == -1) {
17                     ids[u] = current_id;
18                     stack.push_back(u);
19                 }
20             }
21             ++current_id;
22         }
23         return ids;
24     }

```

Listing 4: `connectivity.hpp`: `component_ids()` weist jedem Knoten direkt eine ID zu, ohne die vollständige Komponentenliste zu bauen — effizienter bei reiner ID-Abfrage.

1.3 Tests

Für jeden meiner Algorithmen gibt es eine eigene Testdatei mit mehreren Testfällen, die auch Randfälle abdecken (Einzelknoten, Zyklen, isolierte Knoten, nicht-erreichbare Knoten). Catch2 v3 wird via CMake `FetchContent` automatisch heruntergeladen.

Testdatei	Testfälle (Auswahl)
test_bfs.cpp	Startknoten, Levelreihenfolge, Einzelknoten, Zyklus, Disconnect
test_dfs.cpp	Startknoten, alle Knoten, Einzelknoten, Zyklus, Disconnect
test_topological_sort.cpp	Linearkette, Abhängigkeiten, Einzelknoten, Zyklus
test_connected_components.cpp	2 Komponenten, verbunden, isolierte Knoten, IDs, Einzelknoten

Listing 5 zeigt einen Test, der nicht nur die exakte Reihenfolge prüft, sondern mit `std::find` die relativen Positionen der Knoten überprüft — robuster bei mehreren gültigen topologischen Ordnungen.

```

1 TEST_CASE("Topological sort respects dependencies") {
2     DirectedGraph g;
3     auto v0 = g.add_vertex(); auto v1 = g.add_vertex();
4     auto v2 = g.add_vertex(); auto v3 = g.add_vertex();
5
6     g.add_edge(v0, v2); g.add_edge(v1, v2); g.add_edge(v2, v3);
7
8     auto order = graph::topological_sort(g);
9     REQUIRE(order.size() == 4);
10
11     auto pos = [&](auto v) {
12         return std::find(order.begin(), order.end(), v) - order.
13             begin();
14     };
15     REQUIRE(pos(v0) < pos(v2)); // v0 vor v2
16     REQUIRE(pos(v1) < pos(v2)); // v1 vor v2
17     REQUIRE(pos(v2) < pos(v3)); // v2 vor v3
18 }
```

Listing 5: `test_topological_sort.cpp`: positionsbasierte Prüfung statt exakter Reihenfolge — korrekt, wenn mehrere topologische Ordnungen möglich sind.

Ein weiterer Testfall prüft explizit, dass die topologische Sortierung bei einem zyklischen Graphen eine unvollständige Ausgabe liefert (`order.size() < g.size()`) — eine transparente, beabsichtigte Einschränkung des Kahn-Algorithmus.

1.4 Schwierigkeiten und Grenzen

Bekannte Einschränkungen:

- Topologische Sortierung gibt bei Zyklen keine explizite Ausnahme — nur kürzere Ausgabe als Signal
- `in_degree()` hat Komplexität $O(V+E)$ statt $O(1)$, da keine separate Indegree-Tabelle geführt wird
- Keine Unterstützung für Multigraphen (parallele Kanten)

Die wichtigste technische Entscheidung war der `detail::get_target()`-Helper in `traversal.hpp` und `connectivity.hpp`: Ohne ihn müssten BFS, DFS und Zusammenhangskomponenten

für gewichtete und ungewichtete Graphen doppelt implementiert werden. Mit dem Helper funktionieren alle Algorithmen automatisch für beide Graphtypen (vgl. Listing 3). Die verwendeten Algorithmen (BFS, DFS, Kahn-Algorithmus) sind in [1] ausführlich beschrieben.

1.5 Zwischenstand und Ausblick

Erreichter Stand: Meine Beiträge sind vollständig implementiert und alle 19 Testfälle werden bestanden: `directed_graph.hpp/.cpp`, `undirected_graph.hpp/.cpp`, `traversal.hpp`, `connectivity.hpp`, 4 Testdateien (19 Testfälle total) und 4 Beispielprogramme mit je 3 Szenarien. Die Bibliothek lässt sich als externes CMake-Projekt einbinden und deckt ungewichtete gerichtete sowie ungerichtete Graphen vollständig ab.

Bekannte Grenzen: Die aktuelle Implementierung weist drei konkrete Einschränkungen auf:

- `in_degree()` arbeitet mit $O(V + E)$ statt $O(1)$ — eine separate Indegree-Tabelle würde die Laufzeit verbessern, wurde aber zugunsten einfacherer Datenstrukturen bewusst weggelassen.
- Die topologische Sortierung wirft bei Zyklen keine explizite Ausnahme, sondern signalisiert den Fehler implizit über `order.size() < g.size()`.
- Multigraphen (parallele Kanten) werden nicht unterstützt.

Nächste Schritte: Als konkrete Weiterentwicklungen bieten sich an:

- Erweiterung auf *gewichtete* Graphen: die Header für gewichtete Varianten (`weighted_directed_graph.hpp`, `weighted_undirected_graph.hpp`) sind konzeptionell vorbereitet — der `detail::get_target()` Helper funktioniert bereits für `Edge`-Structs mit `.to`-Feld.
- Hinzufügen von Dijkstra und Bellman-Ford für gewichtete Graphen als natürliche Erweiterung von `traversal.hpp`.
- Einführung einer expliziten Ausnahme (`std::runtime_error`) bei Zyklen in der topologischen Sortierung, um das Verhalten transparenter zu machen.

2 Einschätzung der Gruppenmitglieder

Die folgenden Einschätzungen beziehen sich auf die Zusammenarbeit innerhalb der Gruppe und werden vertraulich behandelt. Der Fokus liegt auf Arbeitsweise, Kommunikation und Kooperation.

Leyla Louache

Rolle & Beiträge: Sprecherin der Gruppe

Als Sprecherin der Gruppe hat Leyla die Organisation der Gruppentreffen übernommen und die Koordination zwischen den Mitgliedern sichergestellt. Sie hat aktiv an den konzeptionellen Diskussionen mitgewirkt — insbesondere bei der Entscheidung für die Adjazenzliste als Datenstruktur und bei der Wahl des Klassendesigns (vier separate Klassen statt einer generischen **Graph**-Klasse). Darüber hinaus hat sie Aufgaben zur Projektstruktur und Dokumentation übernommen.

Stärke: Stets verfügbar und reaktionsschnell

Leyla war während des gesamten Projekts jederzeit erreichbar und hat auf Fragen und Probleme immer schnell reagiert. Dadurch konnten Blockaden im Arbeitsprozess rasch überwunden und Entscheidungen zügig getroffen werden. Ihre Verlässlichkeit hat maßgeblich dazu beigetragen, dass die Zusammenarbeit reibungslos verlief.

Schwäche: Tendenz, zu viele Aufgaben gleichzeitig zu übernehmen

Leyla hat mehrfach Aufgaben auf sich gezogen, die ursprünglich anderen Gruppenmitgliedern zugedacht waren. Das zeigt großes Engagement, führte aber gelegentlich dazu, dass einzelne Teilaufgaben weniger tief bearbeitet wurden als geplant. Eine klarere Aufgabenverteilung wäre hier vorteilhaft gewesen.

Mohamed Eltantawy

Rolle & Beiträge: Algorithmische Unterstützung & Qualitätssicherung

Mohamed hat sich vor allem in den konzeptionellen Phasen des Projekts eingebracht. Er hat die Diskussionen zu den Algorithmen — insbesondere zur Wahl zwischen Kahn-Algorithmus und DFS-Post-Order für die topologische Sortierung sowie zum Unterschied zwischen rekursivem und iterativem DFS — aktiv mitgestaltet. Sein Verständnis der Graphentheorie half dabei, Designentscheidungen wie den `detail::get_target()`-Helper zu hinterfragen und konzeptionell zu festigen. Er hat außerdem Feedback zu den Testfällen gegeben und auf potenzielle Randfälle hingewiesen.

Stärke: Gutes Verständnis der Algorithmen

Mohamed hat die implementierten Graphenalgorithmen schnell durchdrungen und konnte bei konzeptionellen Fragen fundierte Beiträge leisten. Sein algorithmisches Verständnis hat die Qualität der Diskussionen in den Gruppentreffen spürbar verbessert und zur Robustheit der Lösung beigetragen.

Schwäche: Nicht immer verfügbar

Mohamed war gelegentlich schwer erreichbar, was die Koordination innerhalb der Gruppe erschwert hat. Bei zeitkritischen Absprachen — etwa während der Testphase Ende April — fehlte seine Rückmeldung manchmal, sodass Entscheidungen ohne seine Beteiligung getroffen werden mussten.

3 Quellen und Hilfsmittel

Bücher und Artikel

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest und C. Stein, *Introduction to Algorithms*, 4. Aufl. Cambridge, MA: MIT Press, 2022, ISBN: 978-0-262-04630-5. [Referenz für BFS, DFS und topologische Sortierung (Kahn-Algorithmus).]
- [2] A. Hunt und D. Thomas, *The Pragmatic Programmer*. Boston, MA: Addison-Wesley, 1999, ISBN: 0-201-61622-X. [Grundlage für DRY-Prinzip.]
- [3] R. C. Martin, *Agile Software Development, Principles, Patterns, and Practices*. Upper Saddle River, NJ: Pearson, 2002, ISBN: 0-13-597444-5. [Referenz für Single Responsibility Principle.]

Online-Quellen

- [4] cppreference.com, „`std::queue`, `std::stack`, `std::vector`“, *cppreference.com*. [Online]. Verfügbar: <https://en.cppreference.com/w/cpp/container/queue>. [Zugriff: 13. Apr. 2026].
- [5] P. Nash et al., *Catch2*, Version 3.x. [Online]. Verfügbar: <https://github.com/catchorg/Catch2>. [Zugriff: 09. Apr. 2026].
- [6] Kitware Inc., *CMake Documentation*. [Online]. Verfügbar: <https://cmake.org/cmake/help/latest/>. [Zugriff: 09. Apr. 2026].

Einsatz generativer KI

Generative KI (Claude, Anthropic) wurde für folgende Zwecke eingesetzt. Es wurde kein C++-Quellcode direkt generiert.

- **Code-Verfeinerung und Fehlerbehebung:** KI wurde eingesetzt, um bestehenden selbst geschriebenen Code zu überprüfen, Implementierungsfehler zu identifizieren und Verbesserungsvorschläge zu erhalten. Beispielhafte Prompts: „Warum gibt mein iterativer DFS manche Knoten doppelt aus?“, „Wie kann ich `get_target()` so schreiben, dass es sowohl für `std::size_t` als auch für einen *Edge*-Struct funktioniert?“. Alle vorgeschlagenen Korrekturen wurden verstanden, angepasst und eigenständig in den Code integriert.
- **Erstellung des e-Portfolios in L^AT_EX:** KI wurde zur Unterstützung bei der Erstellung des vorliegenden Portfolios verwendet — insbesondere bei der Strukturierung des Dokuments, der Formatierung von Code-Listings, der Gestaltung der Titelseite sowie der Formulierung einzelner Abschnitte auf Deutsch. Beispielhafte Prompts: „Erstelle ein L^AT_EX-Logbuch für mein C++-Graphenprojekt“, „Formuliere die Zusammenfassung des Logbuchs für den Abschnitt über *traversal.hpp* auf Deutsch“.
- **Git-Versionierung und Repository-Verwaltung:** KI wurde zur Unterstützung bei Git-Befehlen eingesetzt, insbesondere beim Einrichten des Repositories, beim Arbeiten mit Branches sowie beim Pushen von Änderungen auf den

JLU-Gitlab-Server. Beispielhafte Prompts: „*Wie pushe ich meinen lokalen Branch auf GitLab?*“, „*Wie löse ich einen Merge-Konflikt zwischen meinem Branch und **main**?*“.

A Logbuch

Das Logbuch ist als Anhang beigelegt und wird nicht auf den Seitenumfang angerechnet. Einträge erfolgen chronologisch. Stichpunktartige Formulierungen sind ausreichend. Verweise auf Gruppentreffen sind durch [GT n] gekennzeichnet.

Eintrag 1 — Erstes Gruppentreffen & Projektplanung

Datum: 09.04.2026

Dauer: 1.5 h

Bezug: [GT 1]

Arbeitsschritte:

- Projektbeschreibung gemeinsam gelesen und besprochen
- Aufgabenverteilung festgelegt: Melchi übernimmt Header-Dateien (`directed_graph.hpp`, `undirected_graph.hpp`, `traversal.hpp`, `connectivity.hpp`), `.cpp`-Implementierungen (`src/`), Tests und Beispielprogramme
- Entscheidung für Catch2 v3 als Test-Framework (via CMake `FetchContent`)
- Git-Repository auf JLU Gitlab eingerichtet, individuelle Branches angelegt
- Ordnerstruktur entworfen: `include/graph/` für Graphklassen-Header, `include/graph/algorithmen/` für Algorithmen-Header, `src/` für `.cpp`-Dateien, `tests/`, `examples/`

Diskussion — Adjazenzliste vs. Adjazenzmatrix:

Zwei Alternativen wurden diskutiert:

	Adjazenzmatrix	Adjazenzliste
Speicher	$O(V^2)$	$O(V + E)$
Kantenzugriff	$O(1)$	$O(\deg(v))$
Nachbar-Iteration	$O(V)$	$O(\deg(v))$
Eignung	dichte Graphen	dünn besetzte Graphen

Entscheidung für Adjazenzliste: Graphenalgorithmen wie BFS und DFS iterieren typischerweise über die Nachbarn eines Knotens. Dünn besetzte Graphen sind in der Praxis häufiger. Die Adjazenzliste ist mit `std::vector<std::vector<vertex_type>>` in modernem C++ direkt verfügbar und effizient.

Diskussion — Klassendesign: eine Klasse oder vier?

Ursprüngliche Idee: eine einzige Klasse `Graph` mit booleschen Parametern `directed` und `weighted`. Problem: bei gewichteten Graphen muss `add_edge()` ein Gewicht entgegennehmen, und `neighbors()` gibt eine andere Datenstruktur zurück (`EdgeStruct` statt `vertex_type`). Das macht eine einheitliche Schnittstelle sehr schwierig.

Entscheidung: vier separate Klassen mit konsistenter Schnittstelle. Duck Typing über C++-Templates ermöglicht, dass alle Algorithmen ohne Vererbung oder gemeinsame Basisklasse mit jedem Graphtyp funktionieren.

Eintrag 2 — Entwurf der Header: `directed_graph.hpp` & `undirected_graph.hpp`**Datum:** 10.04.2026**Dauer:** 2 h**Bezug:** [GT 1]**Arbeitsschritte:**

- Entwurf der öffentlichen Schnittstelle beider Klassen
- Mindestanforderungen: `add_vertex()`, `add_edge()`, `has_vertex()`, `neighbors()`, `size()`, `vertices()`
- Erweiterungen: `has_edge()`, `degree()`, `in_degree()` (nur `DirectedGraph`), `edge_count()`, `empty()`, `edges()`, `to_graphviz()`
- `vertex_type = std::size_t` als typsicherer Alias für Knotenindizes
- Private Datenstruktur: `std::vector<std::vector<vertex_type>>` `adj_`

Diskussion — Warum `to_graphviz()` in der Klasse?

Erste Überlegung: `to_graphviz()` als freie Funktion, analog zu den Algorithmen. Da sie aber direkt auf die interne Adjazenzstruktur zugreift und keine Template-Parameter benötigt, ist sie als Methode sinnvoller. Sie hätte als freie Funktion nur über die öffentlichen Getter funktioniert — mit unnötigem Overhead.

Diskussion — `in_degree()` nur in `DirectedGraph`:

Für ungerichtete Graphen ist `in_degree(v) == degree(v)`, da jede Kante in beide Richtungen gespeichert wird. Die Methode wäre redundant und irreführend. Deshalb wurde sie nur für `DirectedGraph` implementiert.

Diskussion — Rückgabetypp von `neighbors()`:

Zwei Optionen: `const std::vector<vertex_type>` (Kopie) oder `const std::vector<vertex_type>&` (Referenz). Erste Version wurde zunächst implementiert, dann auf Referenz umgestellt: Algorithmen rufen `neighbors()` in jeder Schleifeniteration auf — eine Kopie wäre bei großen Graphen signifikant teurer.

Eintrag 3 — Implementierung: `src/directed_graph.cpp`**Datum:** 11.04.2026**Dauer:** 1.5 h**Bezug:** [GT 1]**Arbeitsschritte:**

- Alle Methoden aus `directed_graph.hpp` implementiert
- `add_vertex()`: `adj_.emplace_back()` statt `push_back()` — effizienter, kein Kopieren eines leeren Vektors
- `has_edge()`: lineare Suche in `adj_[from]` — ausreichend für die erwarteten Graphgrößen
- `in_degree()`: Iteration über alle Adjazenzlisten, $O(V + E)$
- `edge_count()`: Summe aller Listengrößen
- `to_graphviz()`: Ausgabe als `digraph { from -> to; }`

```

1 DirectedGraph::vertex_type DirectedGraph::add_vertex() {
2     adj_.emplace_back();

```

```

3     return adj_.size() - 1;
4 }
5
6 std::size_t DirectedGraph::in_degree(vertex_type v) const {
7     if (!has_vertex(v)) throw std::out_of_range("Invalid vertex");
8     std::size_t count = 0;
9     for (const auto& neighbors : adj_)
10         for (auto u : neighbors)
11             if (u == v) ++count;
12     return count;
13 }

```

Listing 6: directed_graph.cpp: add_vertex() mit `emplace_back` und `in_degree()` durch vollständige Iteration.

Problem — Ausnahme oder stilles Scheitern?

Diskutiert wurde, ob `neighbors(v)` bei ungültigem `v` eine Ausnahme werfen oder einen leeren Vektor zurückgeben soll. Ein leerer Vektor würde Fehler verbergen: ein Algorithmus würde stillschweigend keine Nachbarn finden, anstatt einen Fehler zu melden. Entscheidung für `std::out_of_range` — *fail fast*.

Eintrag 4 — Implementierung: src/undirected_graph.cpp

Datum: 13.04.2026

Dauer: 2 h

Bezug: [GT 1]

Arbeitsschritte:

- Alle Methoden aus `undirected_graph.hpp` implementiert
- `add_edge(a, b)`: beide Richtungen eingetragen — zentraler Unterschied zu `DirectedGraph`
- `edges()`: if (`a < b`) verhindert Doppelzählung
- `edge_count()`: Gesamtgröße aller Adjacenzlisten / 2
- `to_graphviz()`: Ausgabe als `graph { a - b; }`, ebenfalls mit `a < b` für Eindeutigkeit

```

1 std::vector<std::pair<vertex_type, vertex_type>>
2 UndirectedGraph::edges() const {
3     std::vector<std::pair<vertex_type, vertex_type>> result;
4     for (vertex_type a = 0; a < adj_.size(); ++a)
5         for (auto b : adj_[a])
6             if (a < b)
7                 result.emplace_back(a, b);
8     return result;
9 }
10
11 std::size_t UndirectedGraph::edge_count() const {
12     std::size_t count = 0;
13     for (const auto& neighbors : adj_)
14         count += neighbors.size();
15     return count / 2;

```

16 }

Listing 7: `undirected_graph.cpp`: `edges()` gibt jede Kante genau einmal zurück. `edge_count()` korrigiert die doppelte Speicherung.

Problem — `has_edge()` für ungerichtete Graphen:

Erste Implementation suchte nur in `adj_[a]`. Korrekt, da bei `add_edge(a, b)` sowohl `adj_[a]` als auch `adj_[b]` gesetzt werden. Eine Suche in `adj_[b]` wäre redundant.

Eintrag 5 — `traversal.hpp`: `detail::get_target()` Helper

Datum: 15.04.2026

Dauer: 2 h

Bezug: [GT 1]

Problem — Erste Version ohne Helper:

```
1 // FALSCH fuer gewichtete Graphen:
2 for (auto u : g.neighbors(v))
3     if (!visited[u]) { visited[u] = true; q.push(u); }
```

Listing 8: Erster Ansatz (fehlerhaft bei gewichteten Graphen): `u` wird direkt als Knotenindex interpretiert, ist aber ein `Edge-Struct`.

Für ungewichtete Graphen korrekt (`u` ist `std::size_t`), aber für gewichtete Graphen kompiliert das nicht: `u` wäre ein `Edge-Struct`, kein Index. Die Lösung muss zur *Compile-Zeit* entschieden werden.

Lösung — `detail::get_target()` mit Overload Resolution:

```
1 namespace detail {
2
3 template <typename T>
4 auto get_target(const T& neighbor) -> decltype(neighbor.to) {
5     return neighbor.to; // fuer Edge-Structs mit .to-Feld
6 }
7
8 inline std::size_t get_target(std::size_t v) {
9     return v; // fuer ungewichtete Graphen
10 }
11
12 } // namespace detail
```

Listing 9: `traversal.hpp`: zwei Überladungen von `get_target()` — zur *Compile-Zeit* wird die passende gewählt.

Die Template-Überladung greift via `decltype(neighbor.to)` nur wenn `neighbor` ein `.to`-Feld hat. Ansonsten wird die `inline`-Überladung für `std::size_t` gewählt. Dieser Ansatz benötigt keine `if constexpr`, kein `Concept` und keine Vererbung.

Eintrag 6 — traversal.hpp: BFS, DFS, Topologische Sortierung**Datum:** 16.04.2026**Dauer:** 2 h**Bezug:** [GT 1]**BFS — Startknoten vor Schleife markieren:**

Ein subtiles Detail: Der Startknoten wird *vor* dem Einreihen in die Queue als besucht markiert, nicht erst beim Herausnehmen. Würde man erst beim Pop markieren, könnte der Startknoten mehrfach in die Queue geraten (z. B. wenn ein Nachbar auf ihn zeigt).

```

1  visited[start] = true;      // WICHTIG: vor dem Einreihen
2  q.push(start);
3  while (!q.empty()) {
4      V v = q.front(); q.pop();
5      order.push_back(v);
6      for (const auto& neighbor : g.neighbors(v)) {
7          V u = detail::get_target(neighbor);
8          if (!visited[u]) { visited[u] = true; q.push(u); }
9      }
10 }
```

Listing 10: traversal.hpp: BFS mit korrekter Initialisierung. `visited[start] = true` vor `q.push(start)`.

DFS — Rekursiv vs. Iterativ, und wann visited setzen:

Erster Entwurf: rekursiver DFS. Verworfen wegen Stack-Overflow-Risiko bei tiefen Graphen. Zweiter Entwurf: iterativ mit `std::stack`, `visited` beim *Push* setzen. Problem: Knoten können mehrfach in den Stack geraten, bevor sie verarbeitet werden — falsche Ergebnisse.

Endlösung: `visited` beim *Pop* prüfen und dann setzen.

```

1  s.push(start);
2  while (!s.empty()) {
3      V v = s.top(); s.pop();
4      if (visited[v]) continue; // Knoten evtl. mehrfach im
5                               // Stack
6      visited[v] = true;
7      order.push_back(v);
8      for (const auto& neighbor : g.neighbors(v)) {
9          V u = detail::get_target(neighbor);
10         if (!visited[u]) s.push(u);
11     }
```

Listing 11: traversal.hpp: iterativer DFS. `if (visited[v]) continue` beim Pop verhindert Mehrfachverarbeitung.

Topologische Sortierung — Kahn vs. DFS-Post-Order:

	Kahn (BFS)	DFS + Post-Order
Iterativ	ja	nein (oder komplex)
Zykluserkennung	implizit: <code>order.size() < g.size()</code>	explizit via Farbmarkierung
Implementierung	übersichtlich	komplexer

Entscheidung für Kahn: vollständig iterativ, kein Stack-Overflow möglich. Zyklen werden implizit erkannt — das Verhalten ist transparent.

Eintrag 7 — `connectivity.hpp`: drei Hilfsfunktionen

Datum: 17.04.2026

Dauer: 2 h

Bezug: [GT 1]

Arbeitsschritte:

- `connected_components()`: äußere Schleife über alle unbesuchten Startknoten; innere iterative DFS sammelt jede Komponente
- `count_components()`: delegiert an `connected_components()`, gibt nur `.size()` zurück
- `component_ids()`: eigene DFS-Schleife mit Integer-ID statt vollständiger Komponentenliste — effizienter wenn nur die ID gebraucht wird
- `is_connected()`: Sonderfall `empty() → true`; sonst `count_components() == 1`

Diskussion — `count_components()` als Wrapper vs. eigene Implementierung:

`count_components()` delegiert an `connected_components()` und gibt nur `.size()` zurück. Das ist einfacher zu warten, aber nicht optimal: die vollständige Komponentenliste wird aufgebaut, obwohl nur die Anzahl gefragt ist. Für die erwarteten Graphgrößen ist das akzeptabel. Eine eigene Implementierung wäre effizienter, aber redundant (DRY-Verstoß).

Diskussion — `component_ids()` mit eigener Schleife:

Hier wurde bewusst *nicht* auf `connected_components()` delegiert: `component_ids()` baut kein `vector<vector<V>>` auf, sondern schreibt direkt in ein flaches `vector<int>` der Größe `g.size()`. Das spart Speicher und ist cache-freundlicher.

```

1  std::vector<int> ids(g.size(), -1);
2  int current_id = 0;
3  for (V start = 0; start < g.size(); ++start) {
4      if (ids[start] != -1) continue;
5      std::vector<V> stack{start};
6      ids[start] = current_id;
7      while (!stack.empty()) {
8          V v = stack.back(); stack.pop_back();
9          for (const auto& neighbor : g.neighbors(v)) {
10             V u = detail::get_target(neighbor);
11             if (ids[u] == -1) { ids[u] = current_id; stack.
                push_back(u); }
12         }
13     }

```

```
14     ++current_id;  
15 }
```

Listing 12: `connectivity.hpp`: `component_ids()` mit direkter ID-Zuweisung, ohne vollständige Komponentenliste aufzubauen.

Gleicher `get_target()` Helper wie in `traversal.hpp`:

Da `connectivity.hpp` und `traversal.hpp` unabhängig einbindbar sein sollen (kein `#include` des einen im anderen), wurde der Helper in beiden Dateien im `namespace detail` dupliziert. Eine Alternative wäre ein gemeinsamer `detail.hpp`-Header gewesen — für zwei Dateien war der Aufwand aber nicht gerechtfertigt.

Eintrag 8 — Template-Problem: Linker-Fehler bei ausgelagerten Definitionen

Datum: 20.04.2026

Dauer: 1.5 h

Bezug: [GT 2]

Problem:

Erster Versuch: Algorithmen in `traversal.cpp` auslagern, nur Deklarationen in `traversal.hpp`. Beim Kompilieren des Tests:

```
1 undefined reference to 'std::vector<...> graph::bfs<  
    DirectedGraph>(...)'
```

Listing 13: Linker-Fehler beim Auslagern der Template-Definitionen in eine `.cpp`-Datei.

Ursache:

C++ instantiiert Funktions-Templates erst beim Aufruf mit konkreten Typen. Der Compiler muss zum Zeitpunkt der Instanziierung die vollständige Definition sehen — also in der `.cpp`-Datei des Aufrufers, nicht in einer separat kompilierten `traversal.cpp`.

Mögliche Lösungen:

- **Vollständige Definition im Header (gewählt):** einfach, immer korrekt
- `explicit template instantiation`: nur für bekannte Typen möglich — schränkt Flexibilität ein
- `#include` der `.cpp`-Datei im Header: technisch möglich, aber stilistisch falsch und verwirrend

Entscheidung: vollständige Implementierung in `traversal.hpp` und `connectivity.hpp`. Keine `traversal.cpp` oder `connectivity.cpp`.

Eintrag 9 — Unit-Tests: `test_bfs.cpp` und `test_dfs.cpp`

Datum: 25.04.2026

Dauer: 2 h

Bezug: [GT 2]

Arbeitsschritte:

- Catch2 via CMake `FetchContent` eingebunden — kein manueller Download

- `test_bfs.cpp`: 5 Testfälle
- `test_dfs.cpp`: 5 Testfälle

Testfall — BFS-Levelreihenfolge:

Einfache Prüfung (`order[0] == v0`) reicht nicht. Der Test prüft, dass v_3 erst nach v_1 und v_2 kommt (Levelreihenfolge), via `std::find`:

```

1 auto pos_v1 = std::find(order.begin(), order.end(), v1) - order.
    begin();
2 auto pos_v2 = std::find(order.begin(), order.end(), v2) - order.
    begin();
3 auto pos_v3 = std::find(order.begin(), order.end(), v3) - order.
    begin();
4 REQUIRE(pos_v3 > pos_v1);
5 REQUIRE(pos_v3 > pos_v2);

```

Listing 14: `test_bfs.cpp`: positionsbasierte Prüfung der Levelreihenfolge.

Testfall — Zyklus und Disconnect:

"BFS does not revisit vertices in a cycle": Zyklus 0-1-2-0, prüft `order.size() == 3` — jeder Knoten genau einmal.

"BFS does not reach disconnected vertices": isolierter Knoten v_2 , prüft `std::find(..., v2) == order.end()`.

Eintrag 10 — Unit-Tests: `test_topological_sort.cpp` und `test_connected_components.cpp`

Datum: 28.04.2026

Dauer: 2 h

Bezug: [GT 2]

Testfall — Abhängigkeiten (positionsbasiert):

Wenn sowohl $v_0 \rightarrow v_2$ als auch $v_1 \rightarrow v_2$ existieren, können v_0 und v_1 in beliebiger Reihenfolge kommen. Eine exakte Prüfung würde bei einer korrekten, aber anderen Reihenfolge scheitern. Deshalb: positionsbasierte Prüfung.

```

1 auto pos_v0 = std::find(order.begin(), order.end(), v0) - order.
    begin();
2 auto pos_v1 = std::find(order.begin(), order.end(), v1) - order.
    begin();
3 auto pos_v2 = std::find(order.begin(), order.end(), v2) - order.
    begin();
4 REQUIRE(pos_v0 < pos_v2);
5 REQUIRE(pos_v1 < pos_v2);
6 REQUIRE(pos_v2 < pos_v3);

```

Listing 15: `test_topological_sort.cpp`: positionsbasierte Prüfung ist robuster als exakte Reihenfolge.

Testfall — Zyklus liefert unvollständige Ausgabe:

"Topological sort with cycle returns incomplete order" prüft explizit `order.size() < g.size()`. Das ist ein transparenter, dokumentierter Umgang mit der bekannten

Einschränkung des Kahn-Algorithmus — kein Bug, sondern ein bewusst testbarer Randfall.

Testfall — `component_ids()` Konsistenz:

```

1 REQUIRE(ids[v0] == ids[v1]);    // gleiche Komponente
2 REQUIRE(ids[v2] == ids[v3]);    // gleiche Komponente
3 REQUIRE(ids[v0] != ids[v2]);    // verschiedene Komponenten

```

Listing 16: `test_connected_components.cpp`: prüft nicht die genauen IDs, sondern ihre Konsistenz.

Dieser Test ist unabhängig von der Reihenfolge, in der Komponenten-IDs vergeben werden — robuster als ein Test auf konkrete ID-Werte.

Eintrag 11 — Beispielprogramme: je 3 Szenarien

Datum: 02.05.2026

Dauer: 1.5 h

Bezug: [GT 2]

Entscheidung — 3 Szenarien pro Beispiel:

Ein einzelnes Beispiel zeigt nur den Normalfall. Gezielt wurden auch Randfälle eingebaut:

- Zyklen (Zyklus-Graph)
- Isolierte / nicht-erreichbare Knoten (Disconnect)
- Spezialfall (z. B. Zykluserkennung bei Topo-Sort)

Besonderheit `example_topological_sort.cpp`:

Szenario 1 verwendet eine realitätsnahe Kurs-Abhängigkeitsstruktur (`math`, `algo`, `prog`, `project`, `exam`), statt abstrakte Knotennummern. Das macht die Ausgabe leichter interpretierbar.

Szenario 3 demonstriert explizit das Verhalten bei einem Zyklus:

```

1 if (order.size() < g3.size())
2     std::cout << "Incomplete sort: cycle detected ("
3         << order.size() << "/" << g3.size()
4         << " vertices ordered)\n";

```

Listing 17: `example_topological_sort.cpp`: explizite Ausgabe bei Zykluserkennung.

Besonderheit `example_connected_components.cpp`:

Szenario 3 demonstriert `component_ids()` — die einzige Beispielnutzung dieser Funktion in den Beispielprogrammen. Ausgabe pro Knoten: `v0 -> component 0`, `v1 -> component 0`, `v2 -> component 1`.